Academic Year: 2026-2&

Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 15-10-2026**

COURSE NAME: Computer Networks Laboratory **CLASS: T. Y. B.Tech**

NAME: ANAY SHAH
ROLL NO: I112

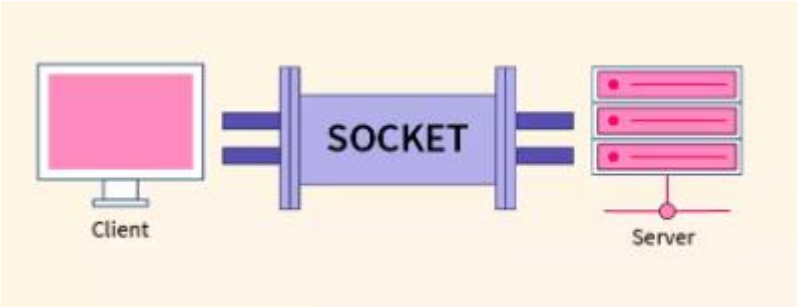
Experiment No. 10

CO/LO: Implement Transport Layer Protocols.

AIM / OBJECTIVE: To Stop and Wait Protocol / Selective Repeat Protocol using Client Server Interaction.

THEORY:

Socket:



Theory:

The Stop-and-Wait Protocol is one of the simplest data link layer flow control and error control mechanisms used in computer networks to ensure reliable data transmission between two connected devices.

Concept

In this protocol, the sender transmits one data frame at a time and then waits for an acknowledgment (ACK) from the receiver before sending the next frame. The receiver, upon receiving a valid frame, sends back an acknowledgment. If the acknowledgment is not received within a specified time (known as the timeout period), the sender retransmits the same frame.

Workflow

1. Sender sends a data frame and starts a timer.
2. Receiver receives the frame, checks for errors, and sends an acknowledgment (ACK) if it is correct.
3. Sender receives the ACK and proceeds to send the next frame.
4. If no ACK is received before timeout, the sender retransmits the same frame.

This mechanism guarantees that frames are delivered in order and with no loss due to transmission errors.

Code:

stop_and_wait_server.py

```
import socket

import random

# Create and configure server socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_socket.bind(('localhost', 12345))

print("Server is up and listening...")

expected_seq = 0

while True:

    data, client_address = server_socket.recvfrom(1024)

    packet = data.decode()


    if packet == 'END':

        print("Transmission ended.")

        break

    seq_num, message = packet.split(':')

    seq_num = int(seq_num)
```

```
    if seq_num == expected_seq:
        print(f"Received: {message} (Seq={seq_num})")
        ack = f'ACK{seq_num}'
        server_socket.sendto(ack.encode(), client_address)
        expected_seq = 1 - expected_seq  # Toggle between 0 and 1
    else:
        print("Duplicate or Out of order packet. Resending ACK.")
        ack = f'ACK{1 - expected_seq}'
        server_socket.sendto(ack.encode(), client_address)
```

stop_and_wait_client.py

```
import socket
import time
import random

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.settimeout(2.0)

server_address = ('localhost', 12345)

messages = ['Hello', 'from', 'Stop', 'and', 'Wait', 'Protocol']
seq_num = 0

for message in messages:
    while True:
        packet = f"{seq_num}:{message}"
```

```
    client_socket.sendto(packet.encode(), server_address)
    print(f"Sent: {message} (Seq={seq_num})")

    try:
        data, _ = client_socket.recvfrom(1024)
        ack = data.decode()
        if ack == f'ACK{seq_num}':
            print(f"ACK received for Seq={seq_num}\n")
            seq_num = 1 - seq_num  # Toggle between 0 and 1
            break
    except socket.timeout:
        print("Timeout. Retransmitting...")
        continue
# Signal end of transmission
client_socket.sendto("END".encode(), server_address)
client_socket.close()
```

If the confirmation doesn't arrive (ACK lost or parcel lost), you resend the same parcel. This analogy represents the Stop-and-Wait mechanism.

Task:

Generating a detailed simulation of packet transmissions with specified loss scenarios and timing events.

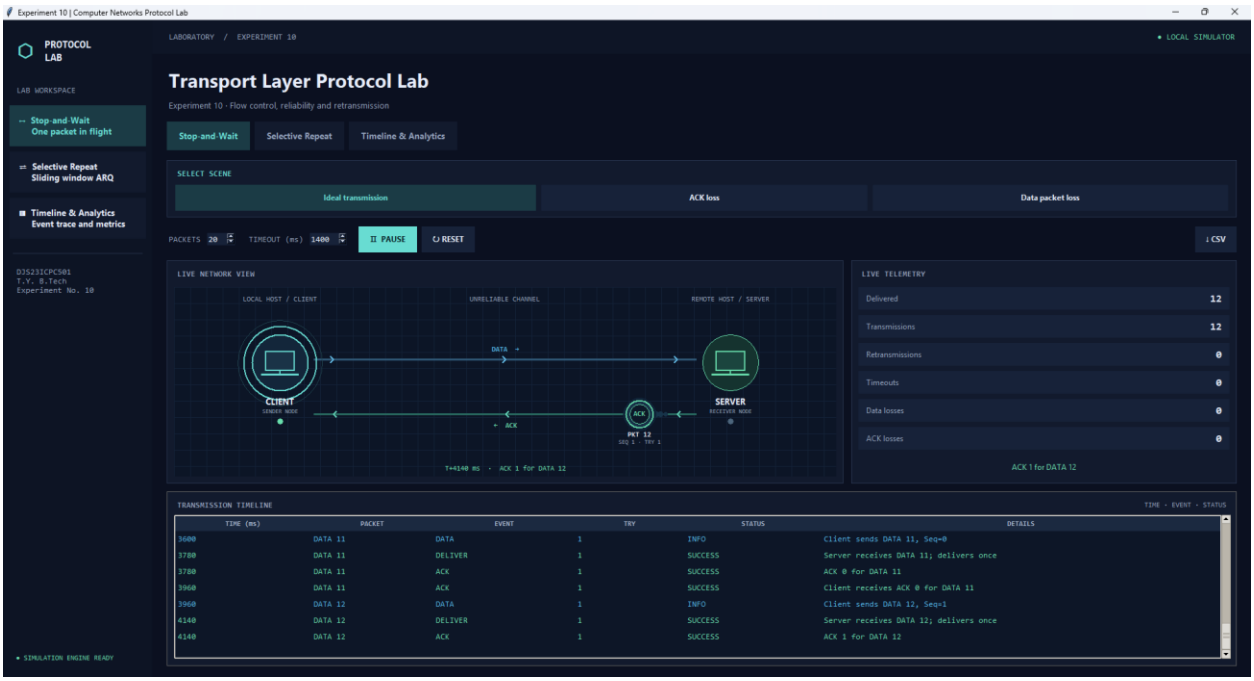
The Stop-and-Wait Protocol simulation for 20 packets with 2 packet losses and 2 ACK losses, showing the timing of transmissions, losses, timeouts, and retransmissions.

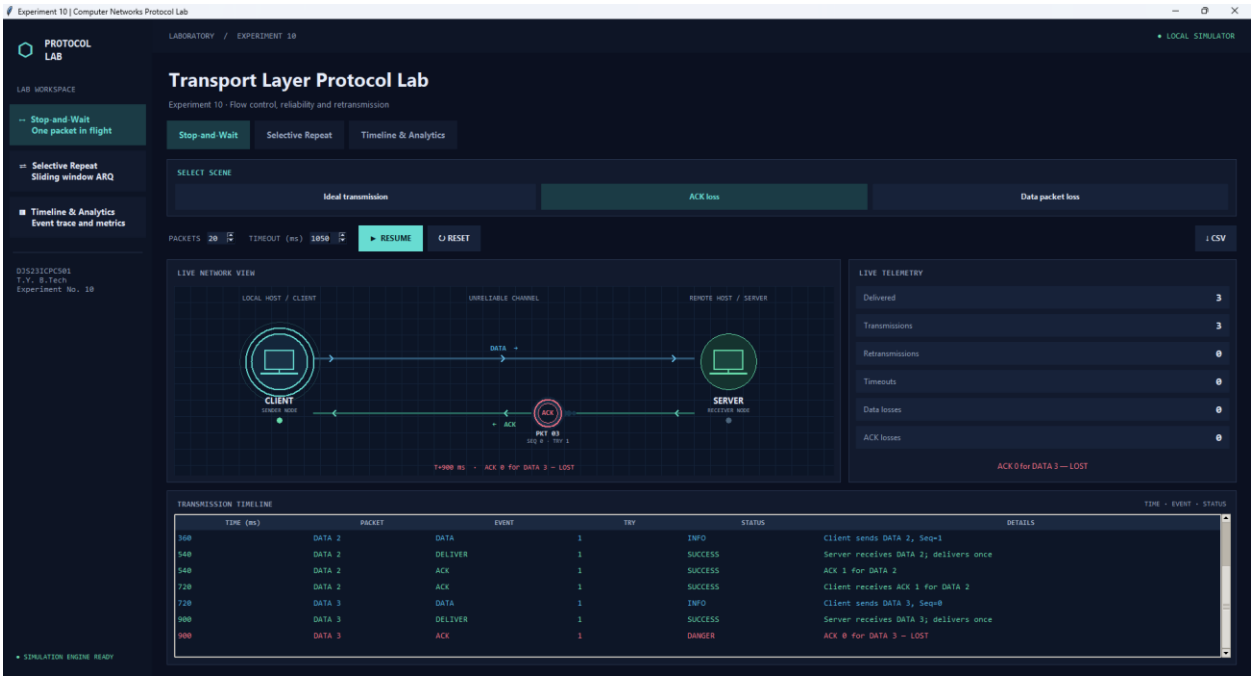
id	time	end	packet	sequence	type	status	message	from	to
1	0	180	1	0	DATA	info	Client sends DATA 1, Seq=0	Client	Server

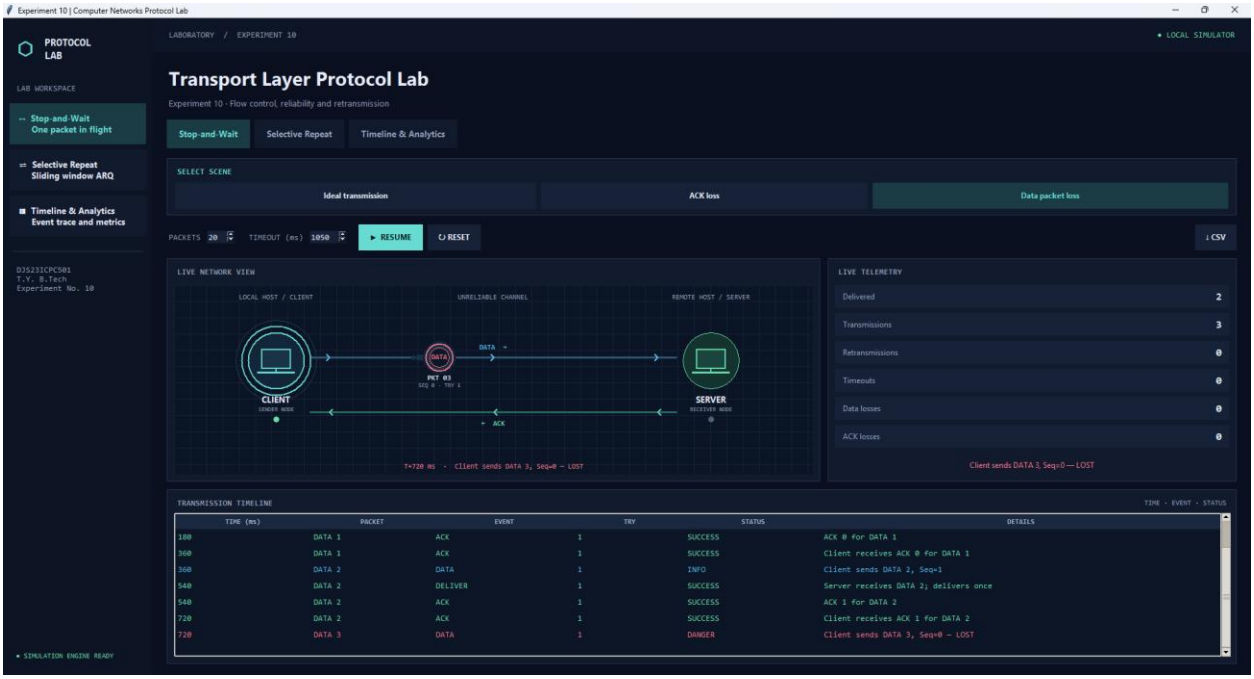
2	180	181	1	0	DELIVER	success	Server receives DATA 1; delivers once	Server	Server
3	180	360	1	0	ACK	success	ACK 0 for DATA 1	Server	Client
4	360	361	1	0	ACK	success	Client receives ACK 0 for DATA 1	Client	Client
5	360	540	2	1	DATA	info	Client sends DATA 2, Seq=1	Client	Server
6	540	541	2	1	DELIVER	success	Server receives DATA 2; delivers once	Server	Server
7	540	720	2	1	ACK	success	ACK 1 for DATA 2	Server	Client
8	720	721	2	1	ACK	success	Client receives ACK 1 for DATA 2	Client	Client
9	720	900	3	0	DATA	info	Client sends DATA 3, Seq=0	Client	Server
10	900	901	3	0	DELIVER	success	Server receives DATA 3; delivers once	Server	Server
11	900	1080	3	0	ACK	success	ACK 0 for DATA 3	Server	Client
12	1080	1081	3	0	ACK	success	Client receives ACK 0 for DATA 3	Client	Client
13	1080	1260	4	1	DATA	info	Client sends DATA 4, Seq=1	Client	Server
14	1260	1261	4	1	DELIVER	success	Server receives DATA 4; delivers once	Server	Server
15	1260	1440	4	1	ACK	success	ACK 1 for DATA 4	Server	Client
16	1440	1441	4	1	ACK	success	Client receives ACK 1 for DATA 4	Client	Client
17	1440	1620	5	0	DATA	info	Client sends DATA 5, Seq=0	Client	Server
18	1620	1621	5	0	DELIVER	success	Server receives DATA 5; delivers once	Server	Server
19	1620	1800	5	0	ACK	success	ACK 0 for DATA 5	Server	Client
20	1800	1801	5	0	ACK	success	Client receives ACK 0 for DATA 5	Client	Client
21	1800	1980	6	1	DATA	info	Client sends DATA 6, Seq=1	Client	Server

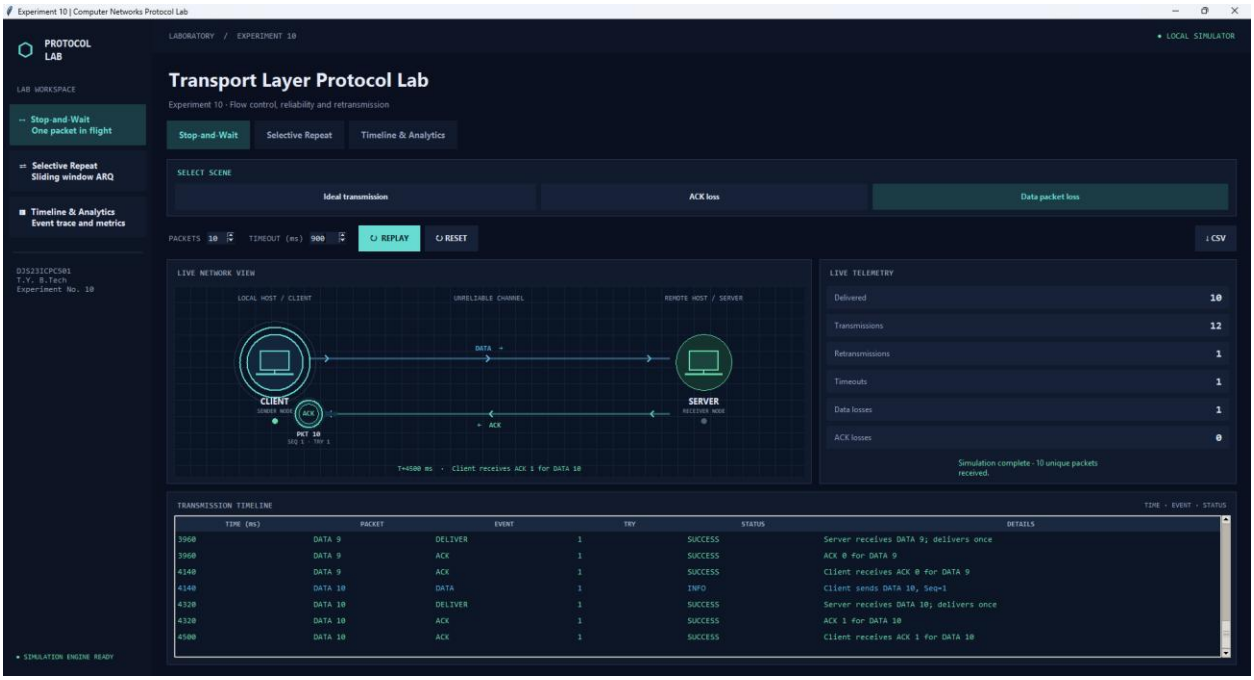
22	1980	1981	6	1	DELIVER	success	Server receives DATA 6; delivers once	Server	Server
23	1980	2160	6	1	ACK	success	ACK 1 for DATA 6	Server	Client
24	2160	2161	6	1	ACK	success	Client receives ACK 1 for DATA 6	Client	Client
25	2160	2340	7	0	DATA	info	Client sends DATA 7, Seq=0	Client	Server
26	2340	2341	7	0	DELIVER	success	Server receives DATA 7; delivers once	Server	Server
27	2340	2520	7	0	ACK	success	ACK 0 for DATA 7	Server	Client
28	2520	2521	7	0	ACK	success	Client receives ACK 0 for DATA 7	Client	Client
29	2520	2700	8	1	DATA	info	Client sends DATA 8, Seq=1	Client	Server
30	2700	2701	8	1	DELIVER	success	Server receives DATA 8; delivers once	Server	Server
31	2700	2880	8	1	ACK	success	ACK 1 for DATA 8	Server	Client
32	2880	2881	8	1	ACK	success	Client receives ACK 1 for DATA 8	Client	Client
33	2880	3060	9	0	DATA	info	Client sends DATA 9, Seq=0	Client	Server
34	3060	3061	9	0	DELIVER	success	Server receives DATA 9; delivers once	Server	Server
35	3060	3240	9	0	ACK	success	ACK 0 for DATA 9	Server	Client
36	3240	3241	9	0	ACK	success	Client receives ACK 0 for DATA 9	Client	Client
37	3240	3420	10	1	DATA	info	Client sends DATA 10, Seq=1	Client	Server
38	3420	3421	10	1	DELIVER	success	Server receives DATA 10; delivers once	Server	Server
39	3420	3600	10	1	ACK	success	ACK 1 for DATA 10	Server	Client
40	3600	3601	10	1	ACK	success	Client receives ACK 1 for DATA 10	Client	Client

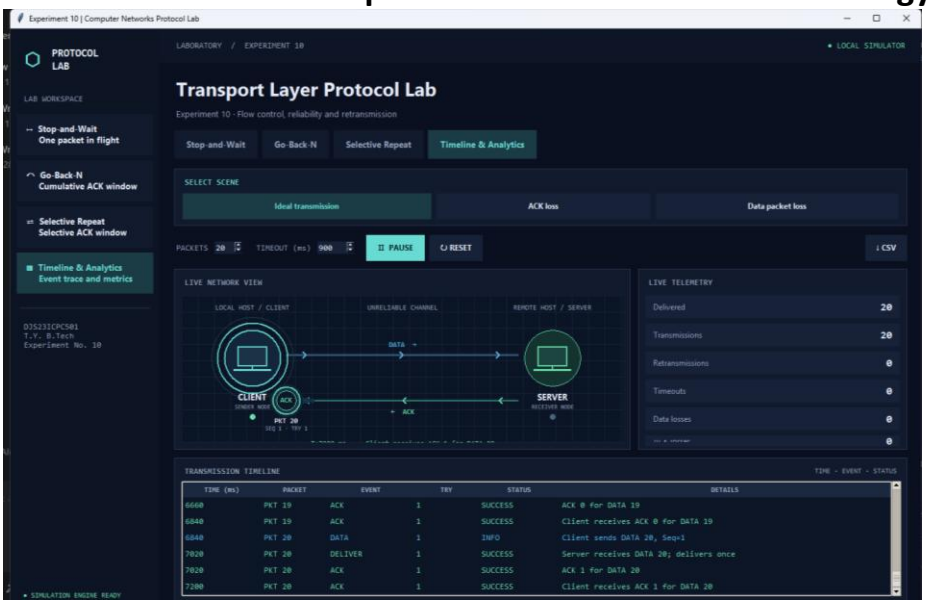
Output:


Experiment 10 | Computer Networks Protocol Lab
PROTOCOL LAB
LABORATORY / EXPERIMENT 10
LOCAL SIMULATOR
Transport Layer Protocol Lab
Experiment 10 · Flow control, reliability and retransmission
Stop-and-Wait
Selective Repeat
Timeline & Analytics
SELECT SCENE
Ideal transmission
ACK loss
Data packet loss
PACKETS 20
TIMEOUT (ms) 1400
PAUSE
RESET
CSV
LIVE NETWORK VIEW
LOCAL HOST / CLIENT
UNRELIABLE CHANNEL
REMOTE HOST / SERVER
CLIENT
SERVER
DATA
ACK
PKT 12
T+4140 ms · ACK 1 for DATA 12
LIVE TELEMETRY
Delivered 12
Transmissions 12
Retransmissions 0
Timeouts 0
Data losses 0
ACK losses 0
ACK 1 for DATA 12
TRANSMISSION TIMELINE
TIME (ms) PACKET EVENT TRY STATUS DETAILS
3600 DATA 11 DATA 1 INFO Client sends DATA 11, Seq=0
3780 DATA 11 DELIVER 1 SUCCESS Server receives DATA 11; delivers once
3780 DATA 11 ACK 1 SUCCESS ACK 0 for DATA 11
3960 DATA 11 ACK 1 SUCCESS Client receives ACK 0 for DATA 11
3960 DATA 12 DATA 1 INFO Client sends DATA 12, Seq=1
4140 DATA 12 DELIVER 1 SUCCESS Server receives DATA 12; delivers once
4140 DATA 12 ACK 1 SUCCESS ACK 1 for DATA 12
SIMULATION ENGINE READY


Experiment 10 | Computer Networks Protocol Lab
PROTOCOL LAB
LABORATORY / EXPERIMENT 10
LOCAL SIMULATOR
Transport Layer Protocol Lab
Experiment 10 · Flow control, reliability and retransmission
Stop-and-Wait
Selective Repeat
Timeline & Analytics
SELECT SCENE
Ideal transmission
ACK loss
Data packet loss
PACKETS 20
TIMEOUT (ms) 1050
RESUME
RESET
CSV
LIVE NETWORK VIEW
LOCAL HOST / CLIENT
UNRELIABLE CHANNEL
REMOTE HOST / SERVER
CLIENT
SERVER
DATA
ACK
PKT 03
T+900 ms · ACK 0 for DATA 3 — LOST
LIVE TELEMETRY
Delivered 3
Transmissions 3
Retransmissions 0
Timeouts 0
Data losses 0
ACK losses 0
ACK 0 for DATA 3 — LOST
TRANSMISSION TIMELINE
TIME (ms) PACKET EVENT TRY STATUS DETAILS
360 DATA 2 DATA 1 INFO Client sends DATA 2, Seq=1
540 DATA 2 DELIVER 1 SUCCESS Server receives DATA 2; delivers once
540 DATA 2 ACK 1 SUCCESS ACK 1 for DATA 2
720 DATA 2 ACK 1 SUCCESS Client receives ACK 1 for DATA 2
720 DATA 3 DATA 1 INFO Client sends DATA 3, Seq=0
900 DATA 3 DELIVER 1 SUCCESS Server receives DATA 3; delivers once
900 DATA 3 ACK 1 DANGER ACK 0 for DATA 3 — LOST
SIMULATION ENGINE READY


Experiment 10 | Computer Networks Protocol Lab
PROTOCOL LAB
LAB WORKSPACE
Stop-and-Wait One packet in flight
Selective Repeat Sliding window ARQ
Timeline & Analytics Event trace and metrics
D3523ICPCS01
T.Y. B.Tech
Experiment No. 10
SIMULATION ENGINE READY
LABORATORY / EXPERIMENT 10
LOCAL SIMULATOR
Transport Layer Protocol Lab
Experiment 10 · Flow control, reliability and retransmission
Stop-and-Wait
Selective Repeat
Timeline & Analytics
SELECT SCENE
Ideal transmission
ACK loss
Data packet loss
PACKETS 20
TIMEOUT (ms) 1050
RESUME
RESET
CSV
LIVE NETWORK VIEW
LOCAL HOST / CLIENT
UNRELIABLE CHANNEL
REMOTE HOST / SERVER
DATA
PKT 03
SEQ 0 · TRY 1
CLIENT
SENDER MODE
SERVER
RECEIVER MODE
ACK
T+720 ms · Client sends DATA 3, Seq=0 — LOST
LIVE TELEMETRY
Delivered 2
Transmissions 3
Retransmissions 0
Timeouts 0
Data losses 0
ACK losses 0
Client sends DATA 3, Seq=0 — LOST
TRANSMISSION TIMELINE
TIME · EVENT · STATUS
TIME (ms) PACKET EVENT TRY STATUS DETAILS
180 DATA 1 ACK 1 SUCCESS ACK 0 for DATA 1
360 DATA 1 ACK 1 SUCCESS Client receives ACK 0 for DATA 1
360 DATA 2 DATA 1 INFO Client sends DATA 2, Seq=1
540 DATA 2 DELIVER 1 SUCCESS Server receives DATA 2; delivers once
540 DATA 2 ACK 1 SUCCESS ACK 1 for DATA 2
720 DATA 2 ACK 1 SUCCESS Client receives ACK 1 for DATA 2
720 DATA 3 DATA 1 DANGER Client sends DATA 3, Seq=0 — LOST


Experiment 10 | Computer Networks Protocol Lab
PROTOCOL LAB
LAB WORKSPACE
Stop-and-Wait One packet in flight
Selective Repeat Sliding window ARQ
Timeline & Analytics Event trace and metrics
D3523ICPCS01
T.Y. B.Tech
Experiment No. 10
SIMULATION ENGINE READY
LABORATORY / EXPERIMENT 10
LOCAL SIMULATOR
Transport Layer Protocol Lab
Experiment 10 · Flow control, reliability and retransmission
Stop-and-Wait
Selective Repeat
Timeline & Analytics
SELECT SCENE
Ideal transmission
ACK loss
Data packet loss
PACKETS 10
TIMEOUT (ms) 900
REPLAY
RESET
CSV
LIVE NETWORK VIEW
LOCAL HOST / CLIENT
UNRELIABLE CHANNEL
REMOTE HOST / SERVER
DATA
CLIENT
SENDER MODE
ACK
PKT 10
SEQ 1 · TRY 1
SERVER
RECEIVER MODE
ACK
T+4500 ms · Client receives ACK 1 for DATA 10
LIVE TELEMETRY
Delivered 10
Transmissions 12
Retransmissions 1
Timeouts 1
Data losses 1
ACK losses 0
Simulation complete - 10 unique packets received.
TRANSMISSION TIMELINE
TIME · EVENT · STATUS
TIME (ms) PACKET EVENT TRY STATUS DETAILS
3960 DATA 9 DELIVER 1 SUCCESS Server receives DATA 9; delivers once
3960 DATA 9 ACK 1 SUCCESS ACK 0 for DATA 9
4140 DATA 9 ACK 1 SUCCESS Client receives ACK 0 for DATA 9
4140 DATA 10 DATA 1 INFO Client sends DATA 10, Seq=1
4320 DATA 10 DELIVER 1 SUCCESS Server receives DATA 10; delivers once
4320 DATA 10 ACK 1 SUCCESS ACK 1 for DATA 10
4500 DATA 10 ACK 1 SUCCESS Client receives ACK 1 for DATA 10



Observation:

1. **Differentiate Stop and Wait, Go Back N and Selective Repeat with examples.**
2. **How to determine optimal timeout values in such protocols**

Conclusion

The experiment demonstrates how ARQ protocols provide reliable data transmission over an unreliable communication channel. Stop-and-Wait uses a simple one-packet-at-a-time approach. Go-Back-N increases transmission efficiency through a sliding window and cumulative acknowledgements, but can retransmit multiple outstanding packets after a loss. Selective Repeat improves retransmission efficiency by acknowledging and retransmitting packets individually while buffering out-of-order packets.